

素检测 and 因子分解

202400460046 邱泽辉

2026 年 5 月 8 日

1. 实现 Miller-Rabin 算法，测试 Miller-Rabin 算法运行效率（2048bit，4096bit）
这道题上次作业做过了，简要带过

Algorithm 1 Miller-Rabin 算法

```
1: Input: 整数  $n$ 
2: Output: 判断  $n$  是否为素数
3: 将  $n - 1$  表示为  $n - 1 = 2^l m$ ，其中  $m$  是奇数
4: 随机选取整数  $a$ ，满足  $1 < a < n - 1$ 
5: 计算  $b \leftarrow a^m \bmod n$ 
6: if  $b \equiv 1 \pmod{n}$  then
7:   return “ $n$  是素数”
8: end if
9: for  $i = 0$  to  $l - 1$  do
10:  if  $b \equiv -1 \pmod{n}$  then
11:    return “ $n$  是素数”
12:  else
13:     $b \leftarrow b^2 \bmod n$ 
14:  end if
15: end for
16: return “ $n$  是合数”
```

C++ 结合 NTL 实现如下:

```
1 #include <NTL/ZZ.h>
2 #include <iostream>
3 #include <chrono>
4 using namespace std;
```

```
5 using namespace NTL;
6 bool MillerRabin(ZZ n) {
7     if(n == 2) return true;
8     if(n < 2 || n % 2 == 0) return false;
9     ZZ m = n - 1;
10    int l = 0;
11    while (m % 2 == 0) {
12        m /= 2;
13        l++;
14    }
15    ZZ a = RandomBnd(n-3) + 2;
16    ZZ b = PowerMod(a, m, n);
17    if (b == 1) {
18        //printf("素数\n");
19        return true;
20    }
21    for(int i = 0; i < l; i++) {
22        if (b == n - 1) {
23            //printf("素数\n");
24            return true;
25        }
26        b = PowerMod(b, 2, n);
27    }
28    //printf("合数\n");
29    return false;
30 }
31
32 void test(ZZ n) {
33     auto start = chrono::high_resolution_clock::now();
34     bool result = MillerRabin(n);
35     auto end = chrono::high_resolution_clock::now();
36     auto duration = chrono::duration_cast<chrono::milliseconds>(end -
37 start);
38     cout << "Miller-Rabin算法判定" << n << "为" << (result ? "素数" : "合
39数") << "耗时" << duration.count() << "ms" << endl;
40 }
41
42 int main() {
43     printf("测试2048bit的素数\n");
```

```
41     ZZ P = GenPrime_ZZ(2048);
42     test(P);
43     printf("测试2048bit的数\n");
44     ZZ N = RandomBits_ZZ(2048);
45     test(N);
46     printf("测试4096bit的素数\n");
47     ZZ P1 = GenPrime_ZZ(4096);
48     test(P1);
49     printf("测试4096bit的数\n");
50     ZZ N1 = RandomBits_ZZ(4096);
51     test(N1);
52     return 0;
53 }
```

分析

随机生成一个整数，它是合数的概率较高，因此在测试中采用随机数来测试合数而素数则是用库函数生成的，但问题是，库函数好像本身就是基于多轮 Miller-Rabin 算法实现的，所以生成的素数在测试中自然会被判定为素数，这似乎有一些循环论证的感觉，不过多轮 Miller-Rabin 算法就可以极大降低误判的概率，可以认为这个素数是可信的

附录：测试结果

实现	位数	判定	时间 (ms)
C++	2048	素数	2
C++	2048	合数	2
C++	4096	素数	13
C++	4096	合数	17

表 1: Miller-Rabin 性能对比

2048 bit 素数测试

```
2039029083347791175449514962424947719798676116289494954682753086635997009705903277086620842337463474
3888041881685333472744882736849523372201187094688815688744794032494461716588087570549607825649176514
8968384426512563039472726340492804517943471268164855250177555060674686344889826161739474366883114679
7148595033667795837191393250297242715686665732209357431033575613694088727963350082994629366158467596
0789627499726429436475907787349800057788566991147984984918958869514892670803536520048971075358718350
1914485766025984205203442565506595030663831646466890672256726582839621451571037198854466319685214304
58843869648899149
```

判定结果：素数；耗时：2 ms。

C++：2048 bit 合数测试

```
2326533765438464268159917020650811528851161254918679864745827473410726468987628423035042810415135386
5590768235670249522627185391144843333552219807616584009945691715482836451379321031500121130234022163
2627009758018024658250875618101298277412282509483046053172297093449182375286932141595448477052520330
0418899327788167015916114825607923713174543517996064582507129117352825428583915039520348462070896290
5582809527472264597679956374246798446645919064864715278055145308016187016706043337203717645867427084
7028509776698497088072047486917023853121522103608766798261241514643730817399814650713765013310676574
41862083815373568
```

判定结果：合数；耗时：2 ms。

4096 bit 素数测试

```
8904653566152935026435485022020367092625337191432224016910242963969686170611197503506612098661387240
5879904760839283742454310291971076311510332595596656979272321387822285710991692306244266324066737997
9918805049477421422899039493315985225988290239966055210085617240009552675230877001958717303701677054
0093424687956771422974926079297412504705899804102718985881322281986760837487210808471460969793576510
1132296544921933864636119572373530844779179525265645505079684606692872036864305443592689234650319927
3740798078424174761731331751858027462168436016142195463910202615662660239381131910260991624602769200
1948953244071696947637445142172682806062795063080209826018836303303161336590415272261341027902768898
5708318169733168738504340420622382506478139815232429234323635546138665107192550712232679655035688645
5311876755496839499428704995541953969853875056558423708209731444855309652217413574740928108688663841
6724177197141607617465907429937070984202014620314841791046670208894015448115530331498359309715884343
6620335144502636692788579706965809519525858389092704685997005070703522755577705840391848843887385782
5946005214338604354490958019802777507960365069675312951394159298225179180470338199393004413775046125
507674184200534603875154816871791
```

判定结果：素数；耗时：13 ms。

4096 bit 合数测试

```
8855600210937835133056877034991371019895213310999185081313262064695699713347119617727345824034435219
5237399496190900714703159592551210478309964746553613846609974132348224251252669992352890801541020911
7916226992307227088335739642680503922310744546364390712646981791745874518417723448947338724762120818
9347412582605496030486060580164076469369973057009862590295373608381151956443442437852808029943204748
```

```

7421680826816290632371262368388785199429161509091932329457882940726884290072810848575124983992319516
8181352324088519535334893513928184077120873045684059217903877339054187281073505159282962477364527041
3880216164817563752953716073347843883277803313311560418899249015138006634873036546191538327552196846
2229314273066642733371557297733793239053291353544305696096645527537673269463859702041726865241978137
7250850928245199412431609772843183094245459258827818182589540875051121860232978328331765460325896439
4954644975248918016362053642344040636751372372012921259960800961367312764251317891527163578179603083
7330136813456205072329907643613035580377165753191552854491015679969714721971078614240603990121161635
2720360994245146930618182771540742183458991045391758790874519387681228591429699101837476787074993470
617508116790215823910398568586434

```

判定结果：合数；耗时：17 ms。

2. (1) 设 $p - q = 2d > 0$, 且 $n = pq$. 证明 $n + d^2$ 是完全平方数。

(2) 设 $n = pq$ 是两个奇素数的乘积, 给定小的正整数 d 满足 $n + d^2$ 是完全平方数。如何利用上述信息分解 n ?

解: (1)

$$n + d^2 = pq + \frac{(p - q)^2}{4} = \frac{4pq + (p - q)^2}{4} = \frac{p^2 - 2pq + q^2 + 4pq}{4} = \frac{(p + q)^2}{4} = \left(\frac{p + q}{2}\right)^2$$

所以 $n + d^2$ 是完全平方数

(2) 我们现在要证明如果 $n + d^2$ 是完全平方数, 那么 d 的值是确定的

设 $n + d^2 = k^2$, 则

$$n = k^2 - d^2 = (k - d)(k + d) = pq$$

又因为 p 和 q 是奇素数, 不妨假设 $p > q$, 所以

$$k - d = q, k + d = p$$

因此

$$k = \frac{p + q}{2}, d = \frac{p - q}{2}$$

或者

$$k - d = 1, k + d = n$$

但这样会解得

$$k = \frac{n + 1}{2}, d = \frac{n - 1}{2}$$

此时题目个人认为表述有些不当, 就是这个 d

如果这个 d 是第一问给定的 $p - q = 2d$, 那么显然没有必要说明 $n + d^2$ 是完全平方数

如果这个 d 不是第一问给定的 $p - q = 2d$, 而是满足 $n + d^2$ 是完全平方数的正整数, 那么就会出现两种情况, 一种是 $d = \frac{p - q}{2}$, 另一种是 $d = \frac{n - 1}{2}$

鉴于题目说的“给定小的正整数 d 满足 $n + d^2$ 是完全平方数”，个人认为这个 d 应该是第一种情况，因为第二种情况是一个平凡的解

所以说如果有一个小的正整数 d 满足 $n + d^2$ 是完全平方数，那么只能满足 $d = \frac{p-q}{2}$

那么我们已知 n 和 d ，就可以求出 p 和 q

$$p = q + 2d, n = pq = q^2 + 2dq$$

解这个二次方程就可以得到 q ，再加上 $2d$ 就可以得到 p

$$p = \frac{2d + \sqrt{4d^2 + 4n}}{2}, q = \frac{-2d + \sqrt{4d^2 + 4n}}{2}$$

3. 以下为长度 100 比特的合数（16 进制），请从中选择一个合数进行因子分解。要求：合数 n 所对应序号需等于本人学号尾号；给出分解后的因子（16 进制）；描述所采用算法，并提供 CPU、时间等关键测试数据。

我的学号尾号是 6，所以我选择第 6 个合数进行分解

$$n = 47F09C8B318A52E3D335A7395CACD2B15E$$

采用 Pollard's Rho 算法

Algorithm 2 Pollard's Rho 算法

```

1: Input: 合数  $n$ 
2: Output:  $n$  的一个非平凡因子
3: 选择一个随机函数  $F(x)$  和一个随机起点  $x_0$ 
4: 初始化  $x \leftarrow x_0, x' \leftarrow x_0$ ,
5: 初始化  $p \leftarrow 1$ 
6: while  $p = 1$  do
7:    $x \leftarrow F(x) \bmod n$ 
8:    $x' \leftarrow F(F(x')) \bmod n$ 
9:    $p \leftarrow \gcd(|x - x'|, n)$ 
10: end while
11: if  $p = n$  then
12:   return “失败，重新选择随机函数和起点”
13: else
14:   return  $p$  // 返回一个非平凡因子
15: end if

```

C++ 结合 NTL 实现如下:

```
1 #include <NTL/ZZ.h>
2 #include <iostream>
3 #include <chrono>
4 #include <vector>
5 #include <string>
6 using namespace std;
7 using namespace NTL;
8 bool MillerRabin(ZZ n) {
9     if(n == 2) return true;
10    if(n < 2 || n % 2 == 0) return false;
11    ZZ m = n - 1;
12    int l = 0;
13    while (m % 2 == 0) {
14        m /= 2;
15        l++;
16    }
17    ZZ a = RandomBnd(n-3) + 2;
18    ZZ b = PowerMod(a, m, n);
19    if (b == 1) {
20        //printf("素数\n");
21        return true;
22    }
23    for(int i = 0; i < l; i++) {
24        if (b == n - 1) {
25            //printf("素数\n");
26            return true;
27        }
28        b = PowerMod(b, 2, n);
29    }
30    //printf("合数\n");
31    return false;
32 }
33
34 ZZ F(ZZ x, ZZ n) {
35     ZZ c = RandomBnd(n-1) + 1;
36     return (PowerMod(x, 2, n) + c) % n;
37 }
```

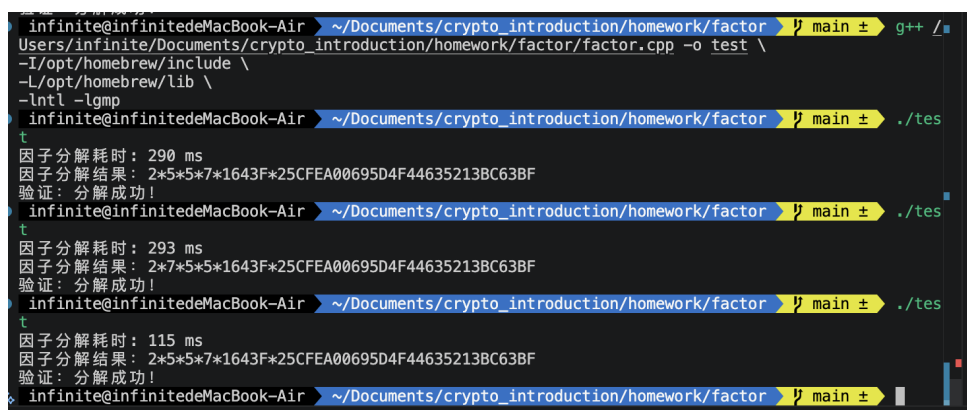
```
38 ZZ Pollard_rho(ZZ n) {
39     if (n <= 1) return n;
40     if (n % 2 == 0) return ZZ(2);
41     if (MillerRabin(n)) return n;
42
43     while (true) {
44         ZZ x = RandomBnd(n - 2) + 2;
45         ZZ y = x;
46         ZZ p = ZZ(1);
47
48
49         while (p == 1) {
50             x = F(x, n);
51             y = F(F(y, n), n);
52             p = GCD(abs(x - y), n);
53         }
54
55         if (p != n) return p;
56
57         // p == n: 本轮失败, 重新随机 x,y,c
58     }
59 }
60 ZZ hex_to_ZZ(const std::string& s) {
61     ZZ res(0);
62     for (char c : s) {
63         res *= 16;
64         if ('0' <= c && c <= '9') res += c - '0';
65         else if ('a' <= c && c <= 'f') res += c - 'a' + 10;
66         else if ('A' <= c && c <= 'F') res += c - 'A' + 10;
67     }
68     return res;
69 }
70 string ZZ_to_hex(const ZZ& n) {
71     if (n == 0) return "0";
72     string res;
73     ZZ temp = n;
74     while (temp > 0) {
75         int digit = to_int(temp % 16);
```



```
76     char c = (digit < 10) ? ('0' + digit) : ('A' + digit - 10);
77     res = c + res;
78     temp /= 16;
79 }
80 return res;
81 }
82 void factor(ZZ n, vector<ZZ>& factors) {
83     if (n <= 1) return;
84
85     // 叶子节点: 素数
86     if (MillerRabin(n)) {
87         factors.push_back(n);
88         return;
89     }
90
91     // 内部节点: 拆成左右子树
92     ZZ p = Pollard_rho(n);
93     factor(p, factors);      // 左子树
94     factor(n / p, factors);  // 右子树
95 }
96 int main() {
97     string s = "47F09C8B318A52E3D335A7395CACD2B15E";
98     ZZ n = hex_to_ZZ(s);
99     vector<ZZ> factors;
100     auto start = chrono::high_resolution_clock::now();
101     factor(n, factors);
102     auto end = chrono::high_resolution_clock::now();
103     auto duration = chrono::duration_cast<chrono::milliseconds>(end -
start);
104     cout << "因子分解耗时:" << duration.count() << "ms" << endl;
105     cout << "因子分解结果:";
106     for (const auto& f : factors) {
107         cout << ZZ_to_hex(f);
108         if (&f != &factors.back()) cout << "*";
109     }
110     cout << endl;
111     ZZ res = ZZ(1);
112     for (const auto& f : factors) {
```

```
113     res *= f;
114 }
115 cout << "验证: ";
116 if (res == n) cout << "分解成功! " << endl;
117 return 0;
118 }
```

分解结果如图



```
infinite@infiniteMacBook-Air ~/Documents/crypto_introduction/homework/factor 主 main ± g++ /...
Users/infinite/Documents/crypto_introduction/homework/factor/factor.cpp -o test \
-I/opt/homebrew/include \
-L/opt/homebrew/lib \
-lntl -lgmp
infinite@infiniteMacBook-Air ~/Documents/crypto_introduction/homework/factor 主 main ± ./tes
t
因子分解耗时: 290 ms
因子分解结果: 2*5*5*7*1643F*25CFEA00695D4F44635213BC63BF
验证: 分解成功!
infinite@infiniteMacBook-Air ~/Documents/crypto_introduction/homework/factor 主 main ± ./tes
t
因子分解耗时: 293 ms
因子分解结果: 2*7*5*5*1643F*25CFEA00695D4F44635213BC63BF
验证: 分解成功!
infinite@infiniteMacBook-Air ~/Documents/crypto_introduction/homework/factor 主 main ± ./tes
t
因子分解耗时: 115 ms
因子分解结果: 2*5*5*7*1643F*25CFEA00695D4F44635213BC63BF
验证: 分解成功!
infinite@infiniteMacBook-Air ~/Documents/crypto_introduction/homework/factor 主 main ±
```

图 1: 因子分解结果

即

$$n = 2 \times 5^2 \times 7 \times 1643F \times 25CFEA00695D4F44635213BC63BF$$

运行三次用时 290, 293, 115 ms, CPU 型号为 Apple M4